ITDBADM

WEEK 10 Class Materials

For today’s session:

- Discuss the remaining classes for this term
- Final Project requirements and deadlines
- Transaction Management
- Check the VM Connections
- All assignments and exams have been graded.
- Please email me if you have questions. If I may have missed your emails, please message me again this week for your concerns.

Remaining Class Schedule

Week	Session	Date	Activity
Week 10	1	March 10 (Tue)	• **Triggers, Transaction Management, Admin Privileges** • **MP Proposal Finalization + Consultation** • **Transaction Management Exercises + Quiz Reviewer** • **Timeslot reservation**
	2	**March 13 (Fri)**	• **Quiz 2: Triggers, Admin Privileges, Transaction Mgmt.** • **Open notes but NO GenAI. Time-bound.**
Week 11	1	March 17 (Tue)	• DB Security (Back-up & Recovery)+ DB App Consultations + Final Proposal
	2	March 20 (Fri)	• **Eid al-Fitr (holiday)**
Week 12	1	**March 24 (Tue)**	• **Submission of MP Presentation and DB Script (ALL GROUPS)** • **Demo**: Presentation (First 3 Groups) - Online
	2	March 27 (Fri)	• **Demo**: Presentation (3 Groups) – F2F
Week 13	1	March 31 (Tue)	• **Demo**: Presentation (3 Groups) - Online
	2	April 3 (Fri)	• **Good Friday (holiday)**
Week 14	1	April 7 (Tue)	• **Demo**: Presentation (2 Groups) – Online or F2F
	2	April 10 (Fri)	• Finals Week (No Final Exam for our Class) • April 15 → Grade Submission + Consultations

Reminder

- **Quiz 2: Friday, March 13**
- **Open Notes Policy**
  - Printed notes or handwritten notes only
  - Prepare your own reviewers
  - **No sharing of notes**
  - **No use of Generative AI**
- **Exam Duration**
  - Strictly **90 minutes**

Final Schedule DB App Demo

Slot Number	Time	March 24 (Tuesday - Online)	March 27 (Friday – F2F)	March 31 (Tue - Online)	April 7 (Tue – F2F/Online)
Slot 1	9:15 AM				
Slot 2	9:45 AM				
Slot 3	10:15 AM				

Note:

- Please be prepared at least 5 minutes before your timeslot. All members should be present and report.
- No need to come to class if not in your presentation timeslot.
- **20-minute demo/presentation + 10-minute Q&A**

Proposal Submission Guidelines (March 17, Tuesday)

- Please prepare and submit the following:
- **Final Proposal (in PDF format, no page limit)**
  Include the following sections:
  - Store Name – A unique and creative name for your online store.
  - Product/Service Description – Brief explanation of what your store sells (e.g., gadgets, clothing, digital services).
  - Target Customers – Identify your main market (e.g., students, professionals, gamers).
  - Key Features – List the planned features of the online store (e.g., multi-currency support, order tracking).
  - Team Members & Roles – Identify each member and their roles (e.g., frontend, backend, database admin).
- **Business Rules**
  - Clearly define your store's operational rules.
  - Try to align them with a specific business scenario that matches your topic
- **Entity Relationship Diagram** (Reverse-Engineered in MySQL Workbench)
- **Note:** The integration of stored procedures and triggers may follow later (not required on Nov 6), once your main schema and business logic are finalized.

Final Project Specifications

- Create a database for a “unique” store.
- **GUI FRONT-END (JAVA, PYTHON, PHP, HTML Any is okay)**
  - Login (based on roles) (At least 3 Roles: Admin, Staff, Customer). Access to specific tables should be controlled.
  - Browse products/ Services (At least 6 product types and/or service types)
  - Currency handling (At least 3 currencies. E.g. PHP, US Dollar, Korean Won)
  - The store should at least have 2 branches.
  - Admin panel (Add/edit products and services)
  - Add to cart and place an order. View order history.
  - Log Tables when using Triggers.
  - Should be directly integrated with MySQL Workbench only! (Do not use MongoDB, PostgreSQL, phpMyAdmin)
- **DELIVERABLES**
  1. **SQL FILES:** Schema, Stored Procedures, Triggers, etc.
     - Should include 6 tables. Justify the relationships of the tables.
     - Should have at least 10 procedures AND 10 triggers each
  2. **GUI Source Codes** (Python, Java, PHP, etc.)
  3. **PPT PRESENTATION** (20 Minute Demo, 10 minute Q&A)
     - PPT Content: ONLINE STORE INTRO, ERD DIAGRAM, DESCRIPTION OF PROCEDURES & TRIGGERS, SCREENSHOTS OF GUI, SQL CODE SNIPPETS, EXAMPLE OUTPUTS
  4. **DEADLINE**: All deliverables should be submitted on **March 24 at 9:15 am (For ALL GROUPS in this class)**

Final Project Rubrics

CRITERIA	Exemplary	Satisfactory	Developing	Beginning	Unacceptable
Grade	4	3	2	1	0
Database Design & Normalization	Database design clearly exceeds the minimum with additional well-normalized tables, enhanced relationships, or advanced business logic	At least 100% of the required core tables are correctly normalized and fully related with PK/FK currencies that reflect proper business rules.	Only 70–99% of the required tables are normalized, or branches/currencies are incomplete	Only 40–69% of the required tables are normalized, or branches/currencies are largely missing or incorrectly related.	Less than 40% of the required tables are normalized, or branches/currencies are missing altogether.
Stored Procedures Implementation	Stored procedures are fully functional, well-documented, and go beyond the minimum in quality, efficiency, or complexity.	At least 100% of the required stored procedures (baseline ≈ 10) are functional and support core transactions.	Only 70–99% of the required procedures are implemented or contain logic errors.	Only 40–69% of the required procedures are implemented or mostly non-functional.	Less than 40% of the required procedures are implemented or functional.
Trigger Implementation	Triggers are fully functional, well-documented, and exceed the minimum in quality, efficiency, or complexity.	At least 100% of the required triggers (baseline ≈ 10) are functional and handle validations/logging.	Only 70–99% of the required triggers are implemented or contain noticeable errors.	Only 40–69% of the required triggers are implemented or are mostly non-functional.	Less than 40% of the required triggers are implemented or functional.
Transaction Management & ACID Compliance	Transactions and logging exceed minimum expectations, demonstrating robust ACID compliance and advanced features.	Minimum transactions are implemented with proper COMMIT/ROLL BACK and accurate logging to meet ACID properties.	About 70–99% of required transactions demonstrate ACID compliance but with some inconsistencies.	About 40–69% of required transactions demonstrate ACID compliance or logs are incomplete.	Less than 40% of required transactions demonstrate ACID compliance or logs are missing.

CRITERIA	Exemplary	Satisfactory	Developing	Beginning	Unacceptable
Grade	4	3	2	1	0
User Privilege Management	Role-based access and security measures exceed the minimum through finer-grained permissions or advanced configurations.	At least 100% of the required roles (baseline ≈ 3 roles: Admin, Staff, Customer) are implemented using GRANT/REVOKE with correct permissions.	Only 70–99% of the required roles or permissions are correctly implemented.	Only 40–69% of the required roles or permissions are correctly implemented.	Less than 40% of the required roles or permissions are correctly implemented.
GUI Design & Usability	GUI is visually appealing, intuitive, and surpasses the minimum with professional layout, responsive design, or enhanced user experience.	GUI meets all minimum functional and design requirements: clean layout, clear navigation, functional login, product browsing, ordering, and currency handling. The Virtual Machine is integrated.	GUI connects to database but layout is cluttered or confusing in at least two areas or only 70–99% of required features work.	GUI is poorly organized or difficult to navigate with only 40–69% of required features functional.	GUI lacks required features or is not usable.
Integration, Presentation & Delivery	Database and GUI integrate seamlessly with zero errors; presentation slides are polished and complete. presenters speak confidently, explain technical details clearly, and engage the audience professionally.	GUI is integrated with database; presentation includes ERD, SQL snippets, screenshots, and a working demo with clear explanations; presenters deliver the report and demo clearly and audibly.	Integration is inconsistent or demo contains noticeable errors; presentation lacks clarity or only 70–99% of required components included; delivery shows limited confidence or clarity.	Integration is unstable and presentation is poorly organized with only 40–69% of required components included; delivery is hesitant or difficult to follow.	No working integration or presentation submitted; students unable to explain the project.
				TOTAL	**28.0**

Virtual Machine

- Make sure you have properly connected your Virtual Machines to your MP. MySQL should be properly integrated.
- Please use the Virtual Machine to set up your DB App. The goal is to have multiple users access your GUI and handle “concurrency”.

Virtual Machine (House Rules)

Each group has its own **Virtual Machine (VM)** connected to MySQL Workbench.
Your VM must be incorporated into your final project to demonstrate database integration.
Follow the rules below to keep the VM secure and running smoothly:

1. Log Off When Not in Use
Always log off from your VM session when you are not actively using it.
This prevents unnecessary resource consumption.

2. Use Your Assigned SSH Port
Connect to MySQL only through the SSH port number assigned to your group.
This ensures proper access control.

3. MySQL Practice & Final Project Only
The VM is provided for MySQL exercises **and your final project work**.
Do not browse the web, install software, or perform non-academic activities.
(CCS Admin monitors usage. They are monitoring the VMs!!!)

4. Protect Your Credentials (Remember your passwords and SSH Ports!)
Keep your assigned username and password private and secure.
You are responsible for managing and safeguarding your login details.

Connecting MySQL to JAVA

1. **MySQL** Workbench is installed and running
2. Make sure you installed **Microsoft** OpenJDK (At least JDK17 or higher)
3. Install **Eclipse IDE** or any IDE to make the GUI
4. MySQL Connector/J JAR file

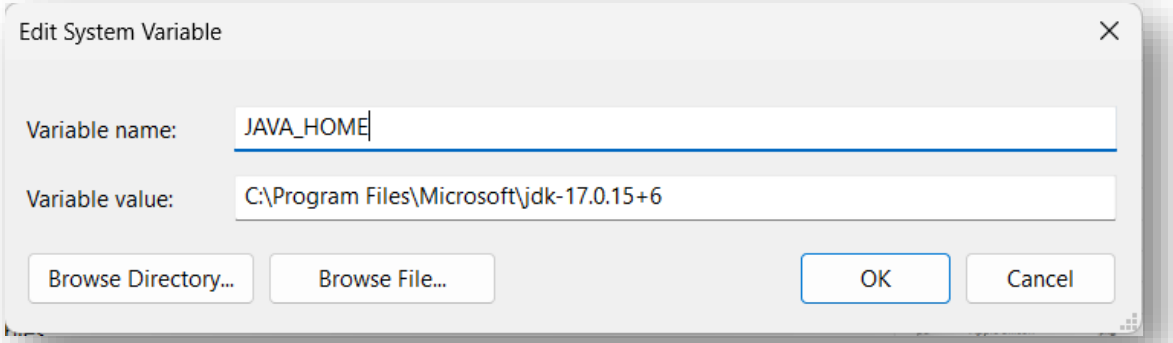
Note:

- Please try to connect MySQL to JAVA or your preferred IDE on your own.
- If you encounter issues, please get in touch with me.

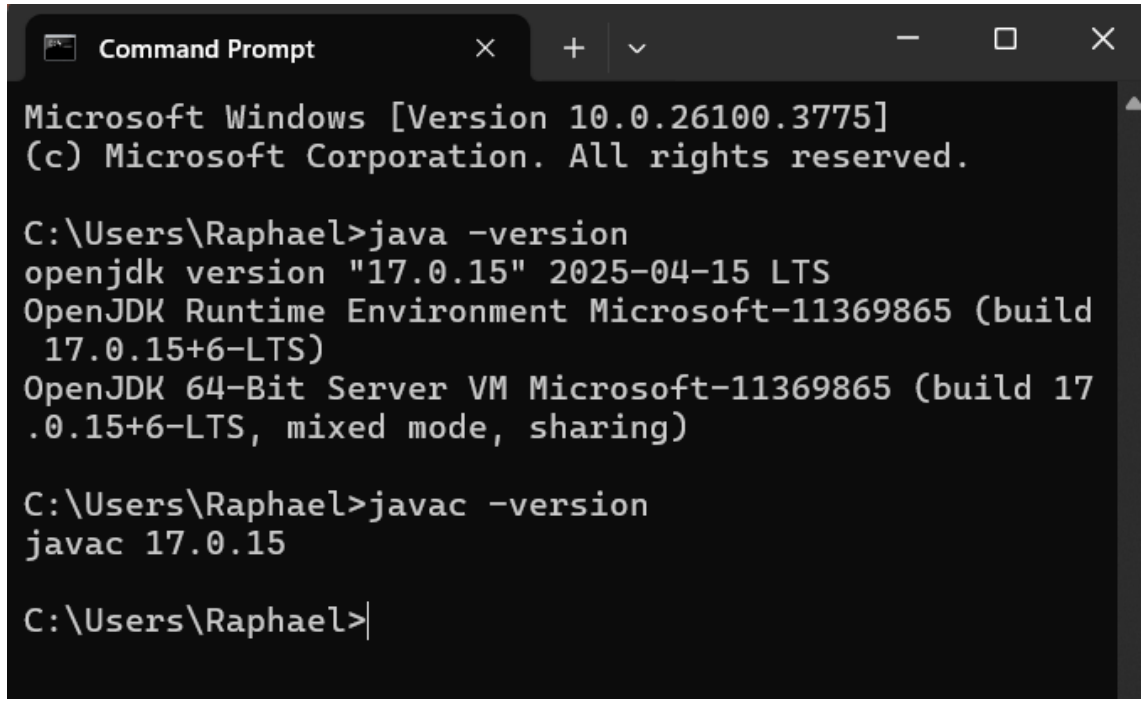
Install JDK (17 or higher)

https://www.oracle.com/ph/java/technologies/downloads

- Make sure Java is installed on your computer. (Check your terminal).
- Different steps for Windows and Mac.
- For windows, make sure Java is properly linked in the Environment Variables.

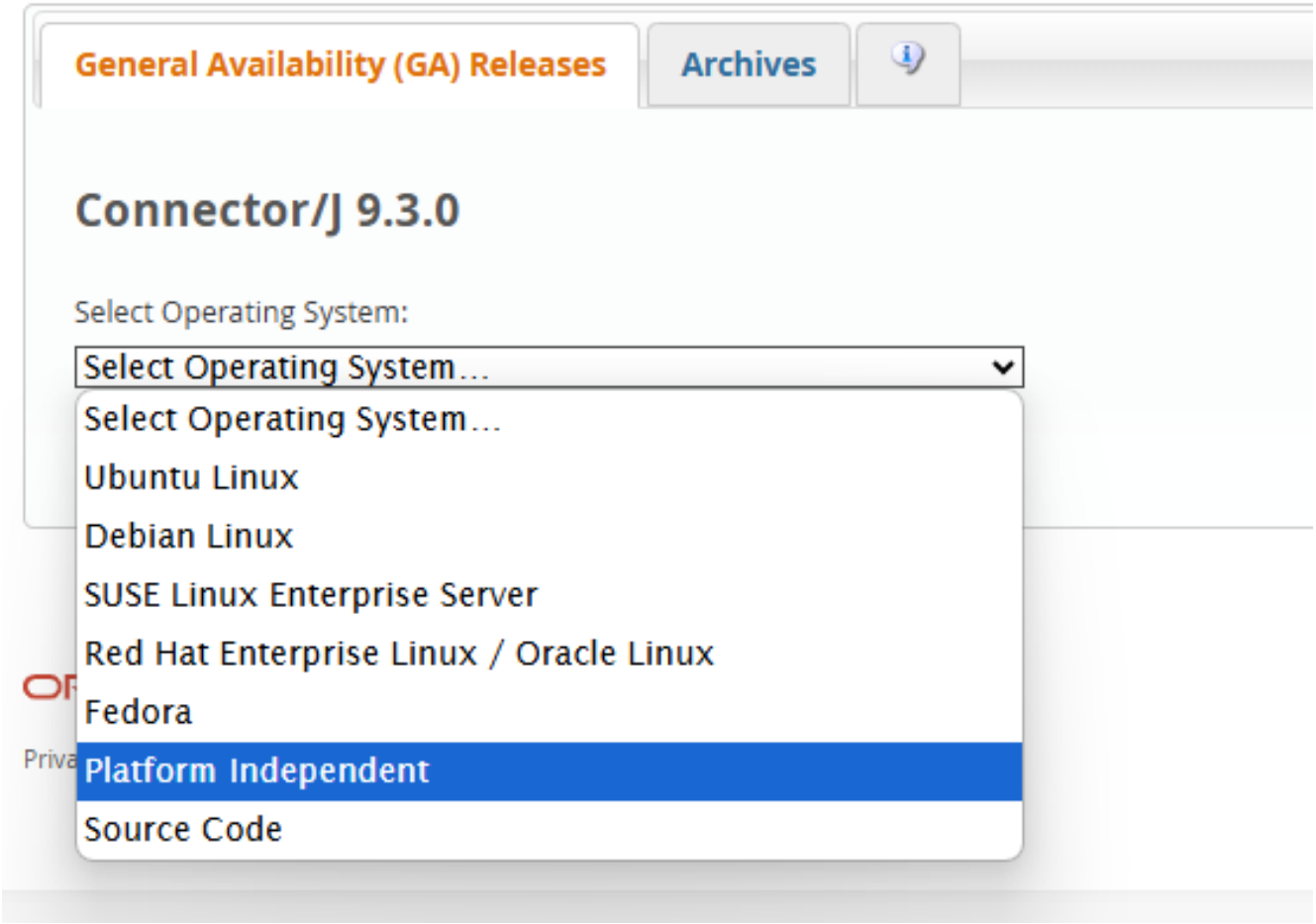


(For windows only)



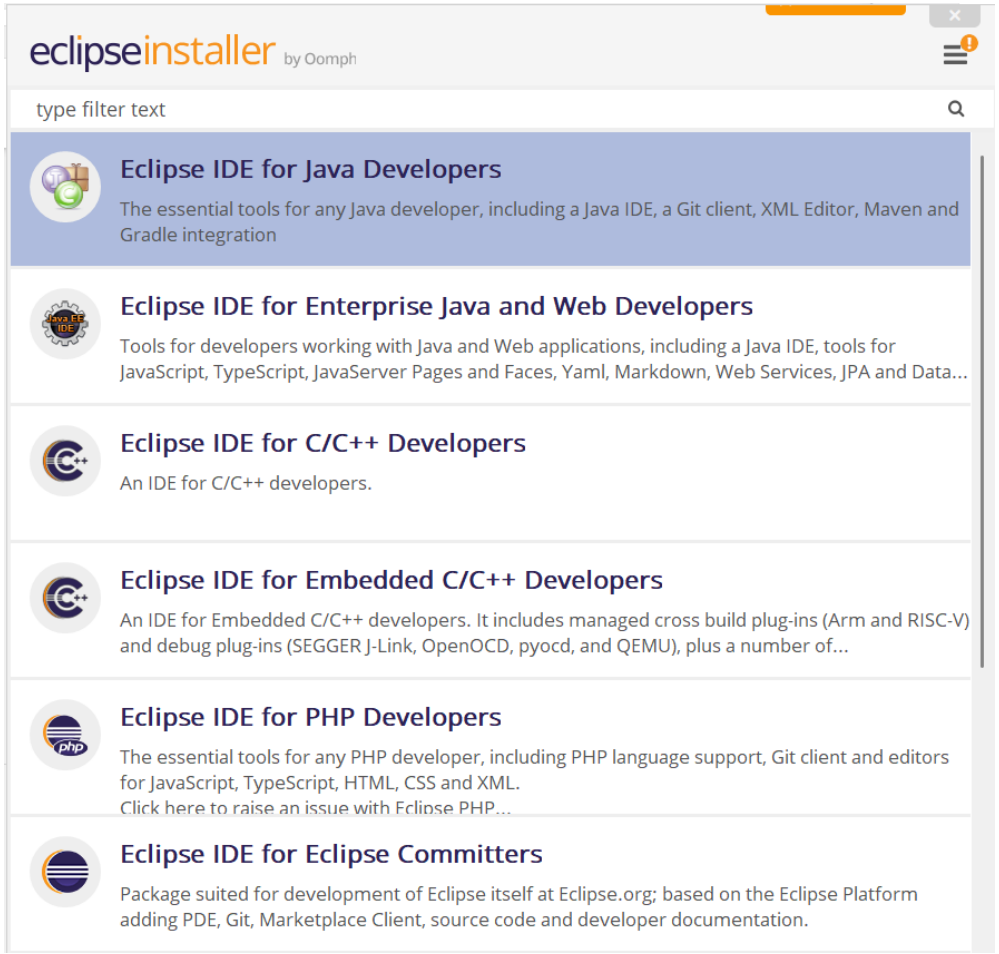
(For windows only)

Install MySQL JDBC (Java Database Connectivity) Driver

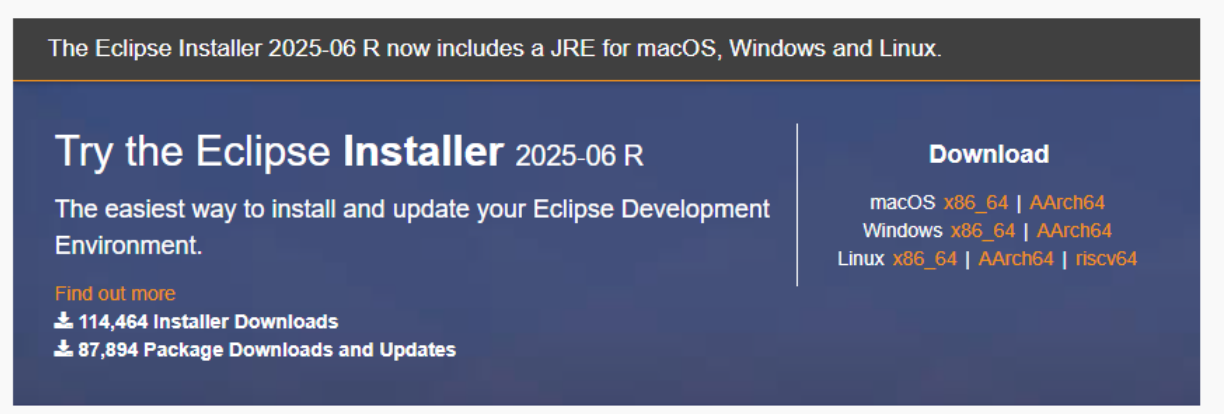


- https://dev.mysql.com/downloads/connector/j/
- Save the file in a secure location to be used in Java

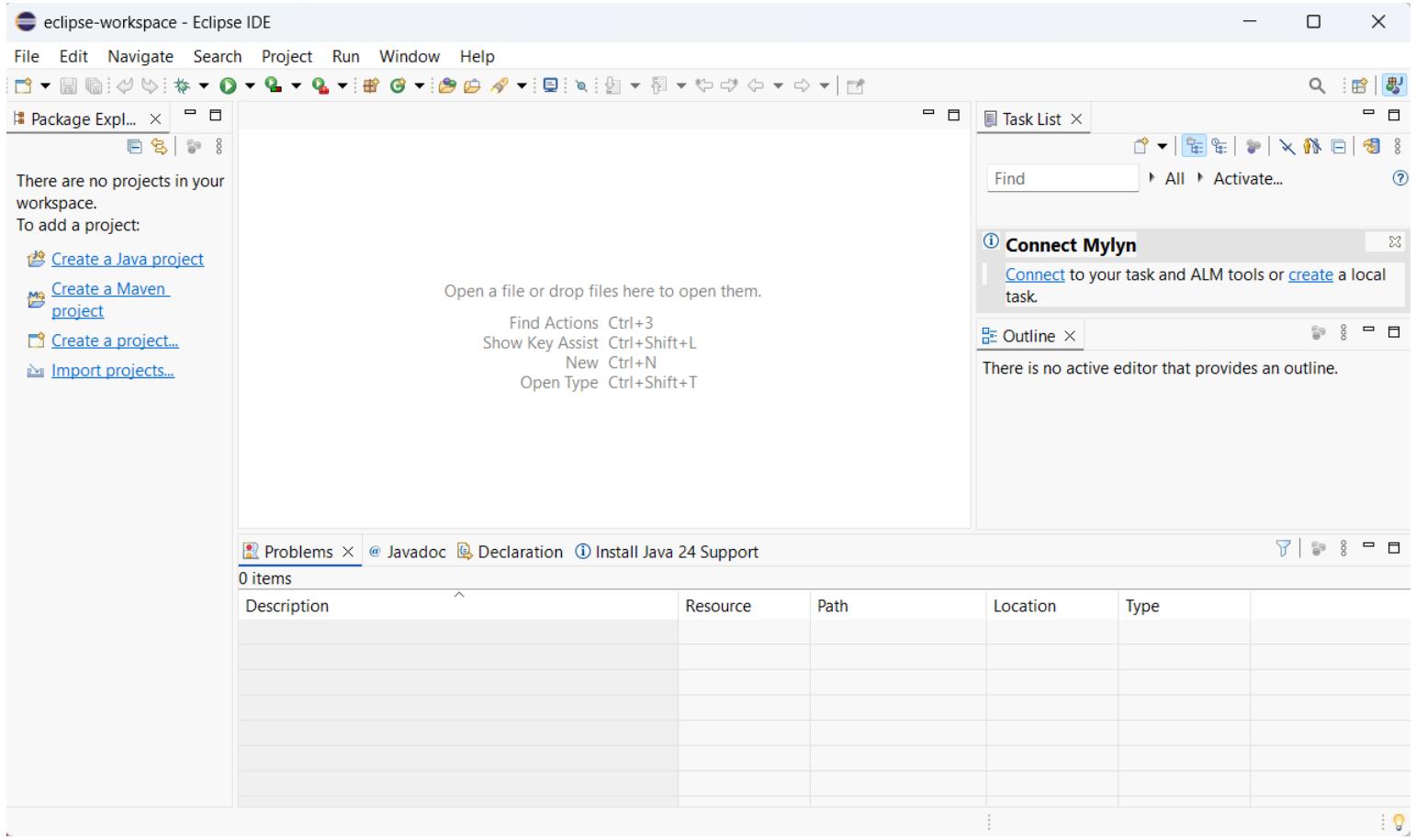
Install Eclipse IDE

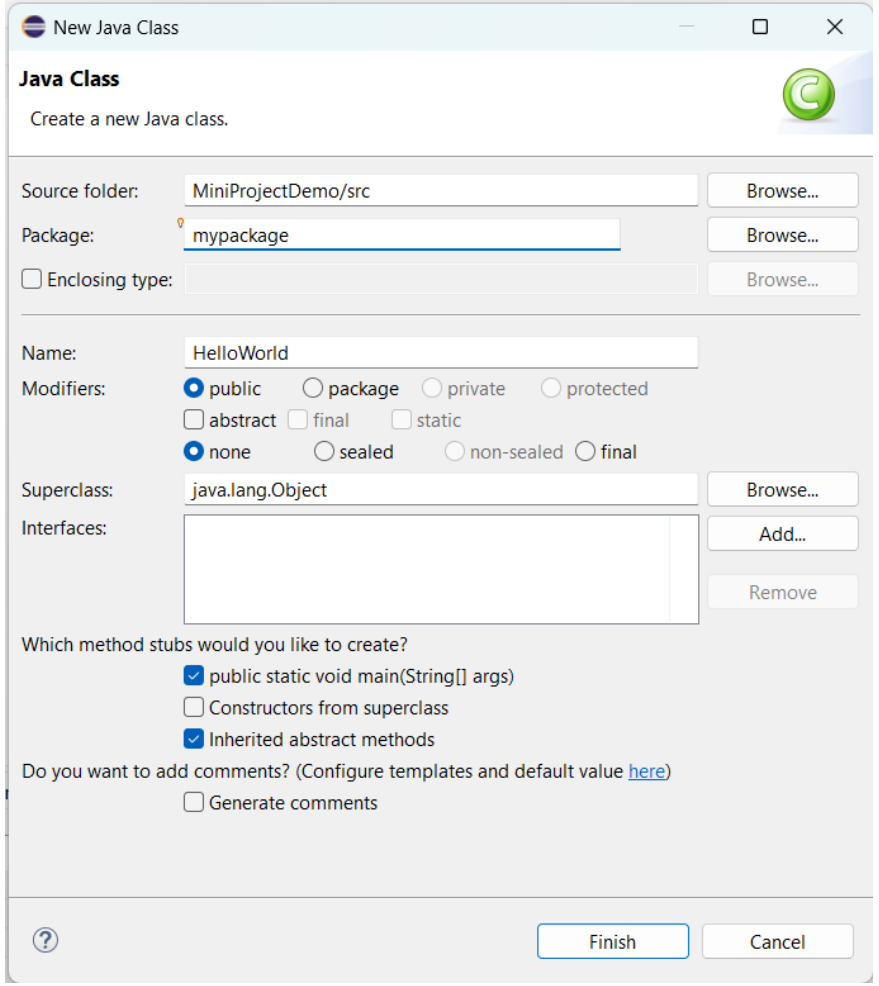


- https://www.eclipse.org/downloads/packages/



Eclipse Interface

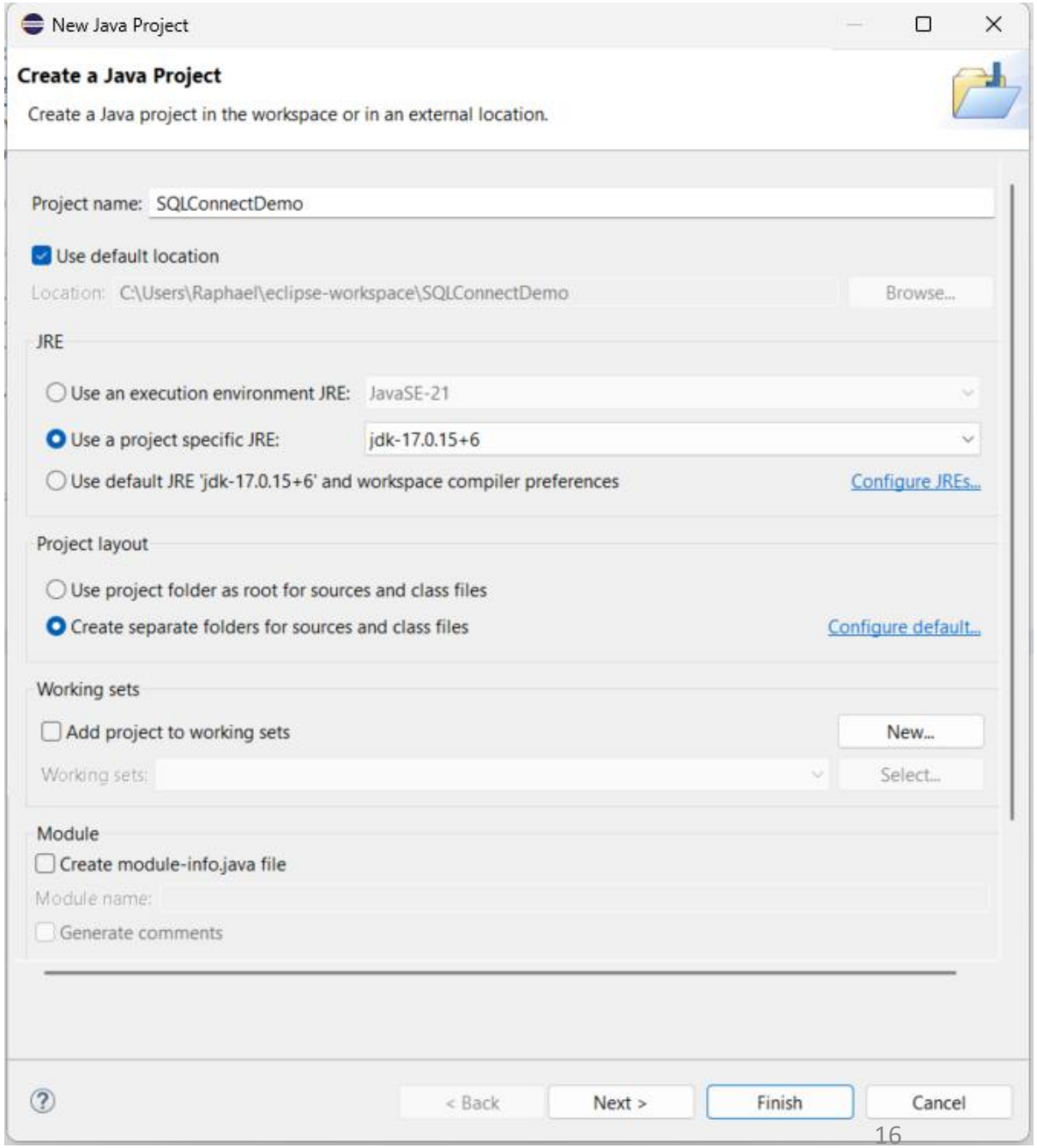




Eclipse IDE

- **Step 1: Create a New Java Project in Eclipse**

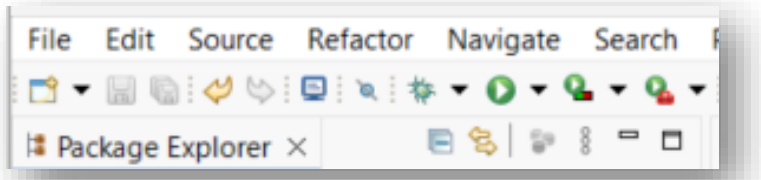
  1. Open Eclipse
  2. Go to File > New > Java Project
  3. Enter project name (e.g., **SQLConnectDemo**)
  4. IMPORTANT: Uncheck "Create module-info.java file"
  5. Click Finish

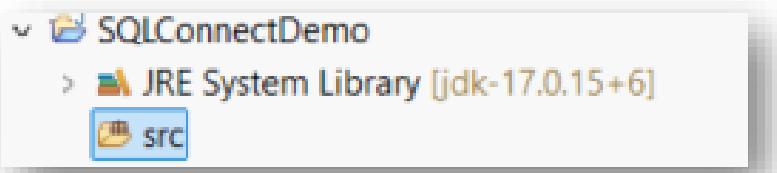


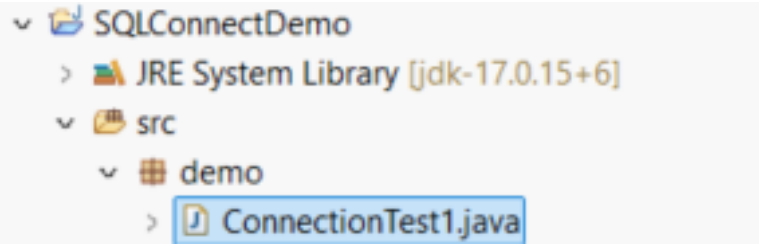
Eclipse IDE

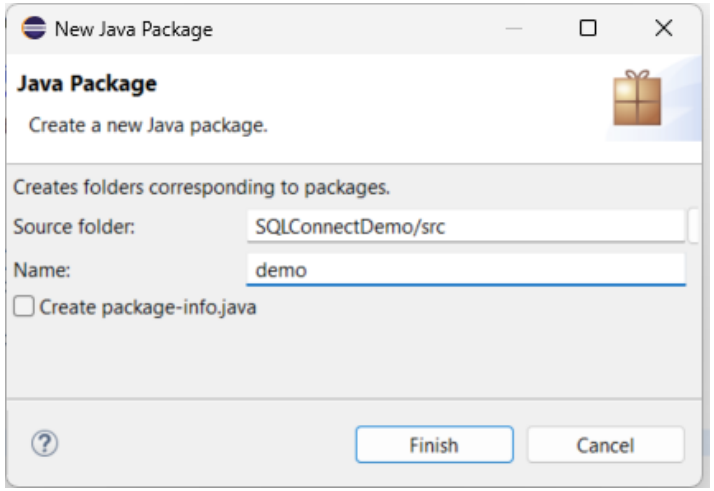
- **Step 2: Create Package and Java Class**

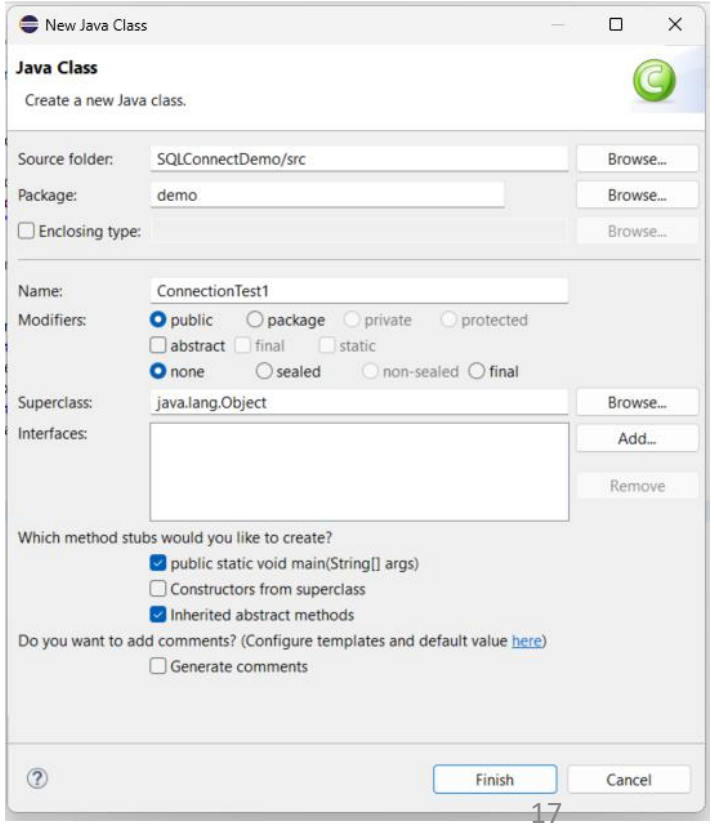
1. Go to Package Explorer in the Left Panel of Eclipse IDE
2. Expand the Java Project (e.g. SQLConnectDemo)
3. Right-click `src` > New > Package > Name it `*demo*` > Finish
4. Right-click the `demo` package > New > Class
   - Name it `*ConnectionTest1*`
   - **Check the box: `public static void main(String[] args)`**
   - Click Finish







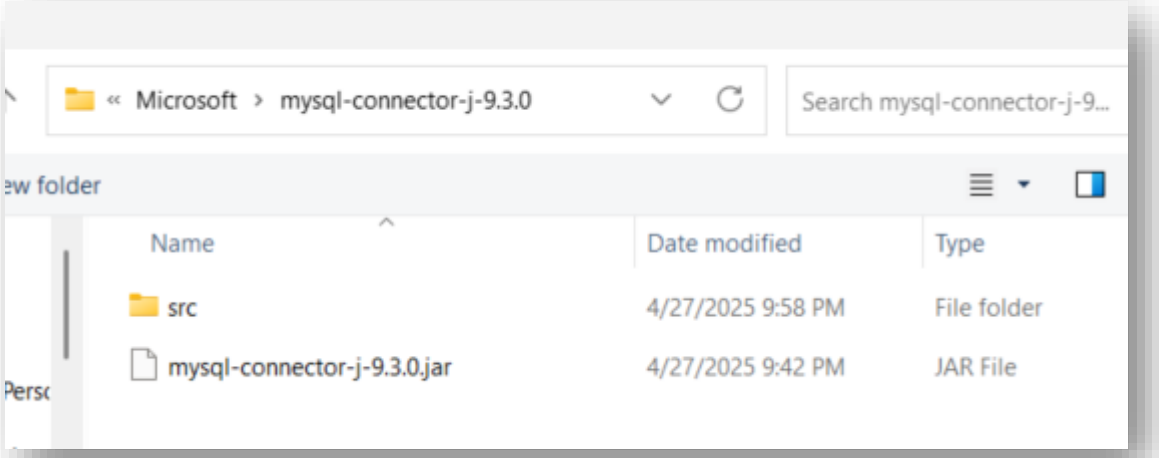


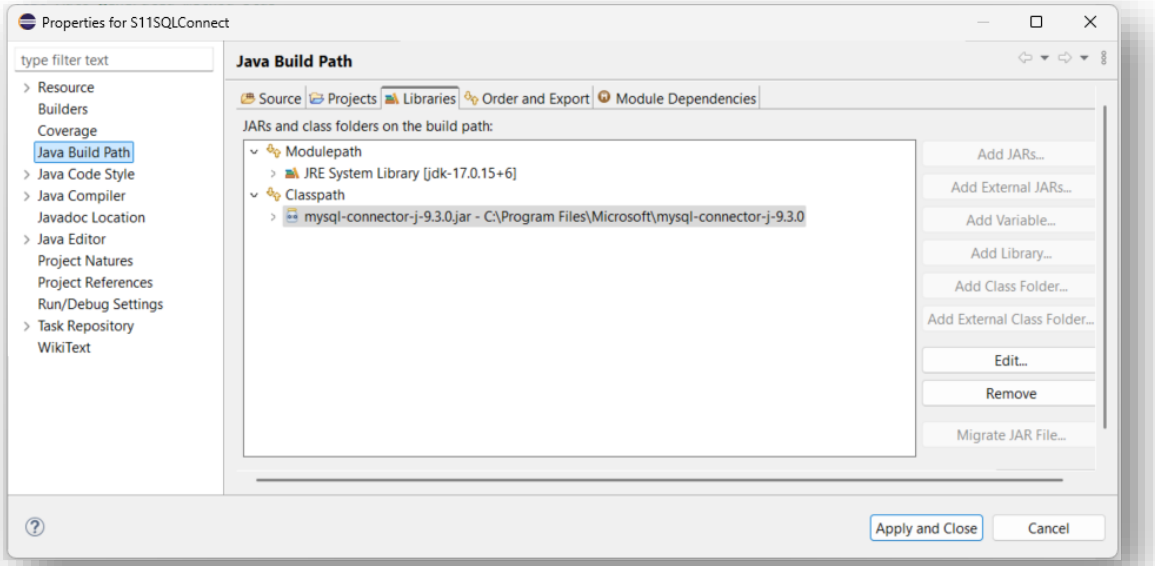


Eclipse IDE

- **Step 3: Add the MySQL JAR to Eclipse**

1. Right-click your project (e.g. SQLConnectDemo) > Build Path > Configure Build Path
2. Go to the Libraries tab > Click Classpath
3. Click Add External JARs
4. Select the `mysql-connector-j-xxx.jar` file
5. Make sure it appears under the **Classpath**, NOT Modulepath
6. Click Apply and Close





Eclipse IDE

- **Step 3: Add the MySQL JAR to Eclipse**

1. Paste the following code to "ConnectionTest1.java"
2. Right-click `ConnectionTest.java`
3. Select Run As > Java Application

```
<terminated> ConnectionTest1 [Java Application] C:\Progr
Successfully connected MySQL
```

```
package demo;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnectionTest1 {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/sakila";
        String user = "root";
        String password = "Dlsu1234!";

        try {
            Connection conn = DriverManager.getConnection(url,
user, password);
            System.out.println("Successfully connected MySQL");
            conn.close();
        } catch (SQLException e) {
            System.out.println("Connection failed.");
            e.printStackTrace();
        }
    }
}
```

Local Connection (Your own computer)

```java
package demo;


import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;


public class ConnectionTest {
public static void main(String[] args) {
String url = "jdbc:mysql://localhost:3306/sakila";
String user = "root";
String password = "Dlsu1234!";


try {
Connection conn = DriverManager.getConnection(url,
user, password);
System.out.println("Successfully connected MySQL");
conn.close();
} catch (SQLException e) {
System.out.println("Connection failed.");
e.printStackTrace();
}
}
}
```

OR

Virtual Machine

```java
package demo;


import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;


public class ConnectionTest1 {
public static void main(String[] args) {
// Use the localhost tunnel from Workbench (Virtual Machine)
String url = "jdbc:mysql://127.0.2.2:3306/sakila"; // <-- change this
String user = "root"; // from your MySQL connection (not SSH username)
String password = "Dlsu1234!"; // replace with the VM password


try {
Connection conn = DriverManager.getConnection(url, user, password);
System.out.println("Successfully connected MySQL");
conn.close();
} catch (SQLException e) {
System.out.println("Connection failed.");
e.printStackTrace();
}
}
}
```

```
<terminated> ConnectionTest1 [Java Application] C:\Progr
Successfully connected MySQL
```

NOTE: MAKE SURE YOUR GUI IS WELL INTEGRATED WITH MYSQL WORKBENCH ON DEMO DAY!!!

ITDBADM

QUIZ 2 REVIEWER

Quiz 2 Guidelines

- **Triggers**
  - Review Practice Exercises (Business Rules, Maintain Audit Log, Delete to Archive)
- **Transaction Management**
  - Acid Properties
  - Concurrency Handling
  - Start Transactions, Commit, Rollback
- **Admin privileges**
  - Grant and revoke privileges

Transactions in SQL

- Every query that runs in a SQL Server is in a transaction
- **Implicit Transaction**
  - A query that does not mention the START TRANSACTION keyword
- **Explicit Transaction**
  - A query that uses START TRANSACTION and ends with COMMIT or ROLLBACK

Transaction Management Syntax

Command	Description
START TRANSACTION;	Begins a transaction block
COMMIT;	Saves all changes made in the transaction
ROLLBACK;	Cancels all changes made in the transaction

Online Store Shopping Cart: Proceed with Order (COMMIT)

```
START TRANSACTION;

UPDATE products
SET stock_quantity = stock_quantity - 1
WHERE product_id = 101;

COMMIT;
```

Online Store Shopping Cart: Cancel Order (ROLLBACK)

```
START TRANSACTION;

UPDATE products
SET stock_quantity = stock_quantity - 1
WHERE product_id = 101;

ROLLBACK;
```

Transaction Management

A **transaction** is a set of SQL operations executed as a single unit.

- Managed using: START TRANSACTION, COMMIT, ROLLBACK
- In SQL transactions, we try to incorporate the ACID properties.

Property	Meaning	Sample
Atomicity	All steps happen or none do	Both inserts happen together or not at all (START + ROLLBACK)
Consistency	Rules are always enforced	If one insert fails, database remains unchanged
Isolation	Separate transactions don't mess with each other	This transaction won't interfere with others until committed
Durability	Once saved, it stays saved. Even if the system fails.	Once committed, changes are permanent, even if server crashes

Transaction Management

- **Atomicity (ALL or Nothing)**
- **Sample Scenario:**
  - You transfer Php 100 from your savings account to your checking account.
  - It should subtract 100 from savings AND add 100 to checking.
  - If anything fails along the way (e.g. system error/server crash/wrong pin), it must UNDO everything.
  - Result: Either both steps happen or neither happens. No partial transfers!!!

Transaction Management

- **Consistency (Rules Must Be Followed)**
  - Data must follow rules. Even after a transaction.
  - When developing a database, business rules should be implemented.
- **Sample Scenario:**
  - Bank rules say accounts can’t have negative balances.
  - If a transfer would leave you with a negative amount *(–Php500)*, the database must reject it.
  - Result: No rule-breaking data allowed. Even after big updates.

Transaction Management

- **Isolation (No interference with others)**
  - Transactions don’t interfere with or see each other’s unfinished work.
- **Sample Scenario:**
  - Two people withdrawing from the same ATM at the same time.
  - Without isolation, both might withdraw Php 500 at the same time. But what if there is only Php 500 left in the ATM. It will cause issues and conflict.
  - With isolation, one transaction waits for the other to finish.
  - Result: Each transaction sees a clean view of the data.

Transaction Management

- **Durability (It stays saved)**
  - Once a transaction is done, it’s saved. Even if the system crashes.
- **Sample Scenario:**
  - A customer deposits Php 5,000 to his savings account.
  - The transaction **commits**.
  - Immediately after, the bank’s server crashes.
  - Result: When the system restarts, the account balance **still reflects** the transfer. The committed transaction is **not lost**, because it was saved to permanent storage before confirming success.

Transaction Management

- **Concurrency Handling**
  - Multiple transactions are happening at the same time. This could lead to conflicts or issues during transactions.
  - It doesn't always mean true "parallel" processing (like multiple CPUs/SQL Workbenches).
  - Instead, it can mean the system *switches quickly* between tasks so they all make progress.
  - Concurrency handling is used to prevent conflicts and ensure data consistency when multiple operations occur at the same time. It typically involves mechanisms like locking, transactions, and isolation levels.
- **Application in databases**
  - Many users can *read* at the same time.
  - BUT… when they *write* (update data), the database has to manage *conflicts* carefully.

Transaction Management

- **Concurrency Handling**
- **Scenario Sample:** Withdrawing money from the bank
- There are two transactions at once

ID	Balance
1	Php 1,000

Transaction A: Withdraw Php 500

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
COMMIT;
```

Transaction B: Withdraw Php 700

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 700 WHERE id = 1;
COMMIT;
```

Without concurrency handling:

- Both transactions will read Php 1000 balance. The last transaction might end up with –Php 200 (Invalid data).

With concurrency handling:

- MySQL can use locks or isolation levels
- When A is changing the balance, B will wait.
- B sees the new balance before withdrawing.
- The database ensures the account doesn’t go below allowed limits.

Transaction Management

- **Concurrency Handling**

1. Basic Transaction Handling (Ensures all steps succeed or all fails)

```
START TRANSACTION;
        UPDATE accounts SET balance = balance - 100 WHERE id = 1;
        UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

Note: If the system fails, use rollback.

```
ROLLBACK;
```

2. Using Locking for Concurrency

```
LOCK TABLES accounts WRITE;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UNLOCK TABLES;
```

3. Implementing Isolation Levels

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
-- your SELECT or UPDATE here
COMMIT;
```

NOTE: THERE ARE 4 LEVELS OF ISOLATION

1. READ UNCOMMITTED: The lowest isolation level.

2. READ COMMITTED: Ensures that only committed changes are visible.

3. REPEATABLE READ: Default isolation level, where repeated reads produce the same results within a transaction.

4. SERIALIZABLE: The strictest level, preventing other transactions from reading or writing the involved data until the transaction is complete.

Admin Privileges (Sakila DB)

- GRANT privileges for Admin and Staff

'username'@'host/ip address' → 'admin'@'%'

```
-- Creating an ADMIN user AND granting full access to the whole database
CREATE USER 'admin'@'%' IDENTIFIED BY ‘Dlsu1234!’; -- Change the name and password
GRANT ALL PRIVILEGES ON *.* TO 'admin'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;

-- Create STAFF user
CREATE USER 'staff'@'%' IDENTIFIED BY ‘Dlsu1234!’;
-- Grant partial access to STAFF (e.g. read-only access to film and actor tables in sakila)
GRANT SELECT ON sakila.film TO 'staff'@'%';
GRANT SELECT ON sakila.actor TO 'staff'@'%';
-- Grant INSERT/UPDATE on a specific table (e.g., rental)
GRANT SELECT, INSERT, UPDATE ON sakila.rental TO 'staff'@'%’;
FLUSH PRIVILEGES; -- To reload the user account info to make changes effective. Clears the cache.

SELECT user, host FROM mysql.user; -- See list of users created in MySQL
SHOW GRANTS FOR 'staff'@'%’; -- Show the privileges for each user (e.g. staff)
```

Admin Privileges (Sakila DB)

- REVOKE privileges for Admin and Staff

```
/* REVOKE privileges for STAFF user */

-- Remove INSERT and UPDATE privileges on sakila.rental
REVOKE INSERT, UPDATE ON sakila.rental FROM 'staff'@'%';

-- Remove SELECT privilege on sakila.actor
REVOKE SELECT ON sakila.actor FROM 'staff'@'%';

-- Remove SELECT privilege on sakila.film
REVOKE SELECT ON sakila.film FROM 'staff'@'%';

-- Optionally, remove all privileges on sakila.rental
REVOKE ALL PRIVILEGES ON sakila.rental FROM 'staff'@'%';

-- Flush to apply changes
FLUSH PRIVILEGES;
```

Practice Exercise

- **INSTRUCTIONS:**

1. Use MySQL Workbench to write and run your triggers.
   The Sakila database will be your data source.
   You will create BEFORE or AFTER triggers depending on the specific task description.
2. Copy your full SQL code. It should include:
   a. CREATE TRIGGER statements
   b. Any CREATE TABLE statements needed for audit or archive tables
   c. Sample INSERT, UPDATE, or DELETE statements you used to test the trigger
3. Show the output results.
   Take a screenshot of the result table(s) showing the effect of the trigger.
   For example, show the audit or archive table after testing your trigger.
   You may also copy-paste the SELECT result output if you don’t want to screenshot.

Trigger Exercise (Sakila Database)

- **Task 1: Enforce Business Rule**
- **Scenario:** You are asked to ensure that no new film can be inserted with a rental rate over $10.
- **Task:** Create a **BEFORE** trigger on the film table.
  If a new film's rental_rate is greater than 10, the trigger should block the insert with an error.
- **Tip:** Use the SIGNAL SQLSTATE syntax to raise an error.
- **Expected Behavior:** Trying to insert a film with rental_rate = 15 should fail with your custom error message.

Task 1 Answer

```
DELIMITER $$

DROP TRIGGER IF EXISTS invalid_rental_rate;

CREATE TRIGGER invalid_rental_rate
BEFORE INSERT ON film
FOR EACH ROW

BEGIN
    IF NEW.rental_rate > 10 THEN
    SIGNAL SQLSTATE '45000’
    SET MESSAGE_TEXT = 'Invalid input. Rental rate must be below 10.';
    END IF;
END

$$ DELIMITER ;
```

```
INSERT INTO film (title, rental_rate) VALUES (‘film1', 15);
```

Trigger Exercise (Sakila Database)

- **Task 2: Maintain Audit Log for Price Change**
- **Scenario:** The company wants to track all price changes for films.
- **Task:** Create a new table called ***film_price_audit*** with these columns:
  - audit_id (auto-increment primary key)
  - film_id (the film's ID)
  - old_price
  - new_price
  - change_date (timestamp)
- Create a trigger on the film table. The trigger should insert a row into film_price_audit only if the rental_rate value was changed.
- Expected Behavior: E.g. When a film's rental_rate is updated from 2.99 to 3.99, a new row is inserted into ***film_price_audit*** with the old and new values.

Task 2 Answer

```
CREATE TABLE film_price_audit(
   audit_id INT AUTO_INCREMENT PRIMARY KEY,
   film_id INT,
   old_price DECIMAL(4,2),
   new_price DECIMAL(4,2),
   change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
DELIMITER $$
DROP TRIGGER IF EXISTS insert_after_change;
CREATE TRIGGER insert_after_change
AFTER UPDATE ON film
FOR EACH ROW
BEGIN
IF NEW.rental_rate != OLD.rental_rate THEN
INSERT INTO film_price_audit (film_id, old_price, new_price)
      VALUES (OLD.film_id, OLD.rental_rate, NEW.rental_rate);
END IF;
END
$$ DELIMITER ;
```

```
UPDATE film
SET rental_rate = 6.99
WHERE film_id = 1;

SELECT * FROM film_price_audit;
```

Note: If you encounter warning messages when deleting or updating tables,
Run the following code to disable the constraints.

```
SET SQL_SAFE_UPDATES = 0;
SET FOREIGN_KEY_CHECKS = 0;
```

Important. Please remember!

Trigger Exercise (Sakila Database)

- **Task 3: Handle Deletes with Archive Table**
- Scenario: The business wants to keep a record of any film that is deleted for auditing purposes.
- Task:
  1. Create a new table called film_archive with appropriate columns to store a deleted film's details (e.g., film_id, title, description, release_year, etc.).
  2. Create a trigger on the film table.
  3. The trigger should copy the deleted film's data into film_archive.
- Expected Behavior: When a film is deleted from film, its data is preserved in the film_archive table.

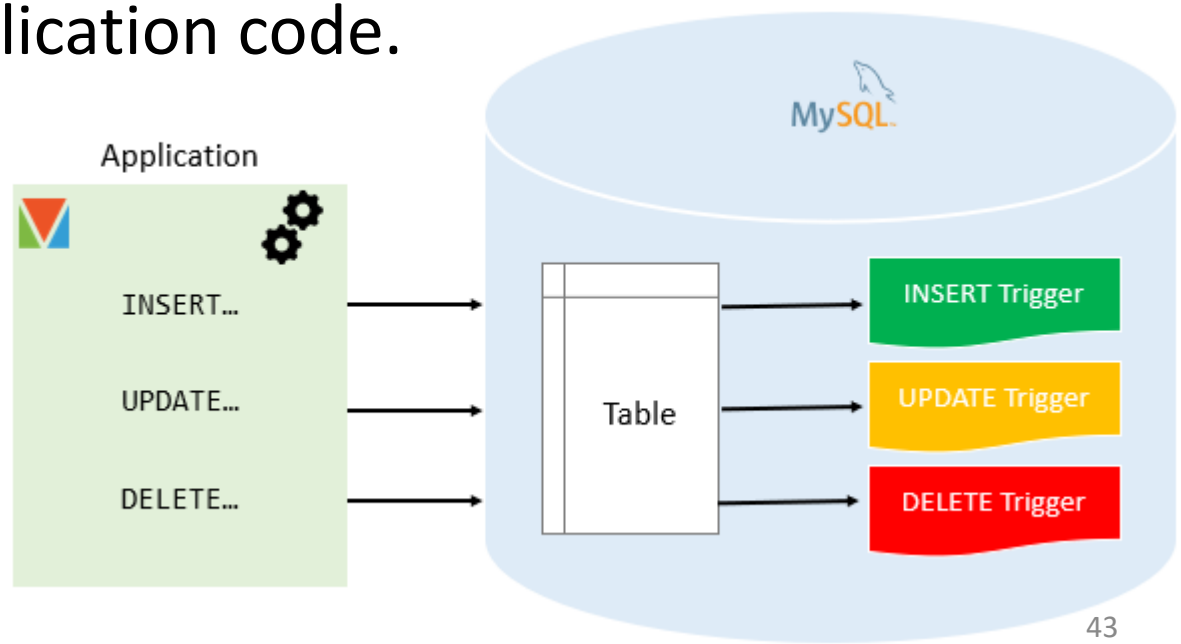
Task 3 Answer

```
CREATE TABLE film_archive (
   archive_id INT AUTO_INCREMENT PRIMARY KEY,
   film_id INT,
   title VARCHAR(255),
   description TEXT,
   release_year YEAR,
   language_id INT,
   rental_duration INT,
   rental_rate DECIMAL(4,2),
   length INT,
   replacement_cost DECIMAL(5,2),
   rating VARCHAR(10),
   special_features VARCHAR(255),
   archived_date TIMESTAMP DEFAULT
  CURRENT_TIMESTAMP
);
```

```
DELIMITER $$
CREATE TRIGGER after_film_delete
AFTER DELETE ON film
FOR EACH ROW
BEGIN
   INSERT INTO film_archive (
      film_id, title, description, release_year, language_id,
      rental_duration, rental_rate, length, replacement_cost,
      rating, special_features
   )
   VALUES (
      OLD.film_id, OLD.title, OLD.description, OLD.release_year,
      OLD.language_id, OLD.rental_duration, OLD.rental_rate,
      OLD.length, OLD.replacement_cost, OLD.rating, OLD.special_features
   );
END;
$$ DELIMITER ;
```

Triggers

- A **trigger** is a set of SQL statements that **runs automatically** when a specific event occurs in a table.
- It is **"triggered"** by actions like **INSERT**, **UPDATE**, or **DELETE**.
- Used to **enforce business rules**, **maintain data integrity**, or **audit changes**.
- Runs **without explicit call** from application code.



Why use triggers?

- **Automate data validation**
  - Enforce rules without manual checks.
- **Maintain consistent data**
  - Ensure integrity across tables.
- **Log or audit changes**
  - Record inserts, updates, deletes.
- **Enforce complex business rules**
  - Automatically react to data changes.
- **Reduce application code complexity**
  - Logic is stored in the database.

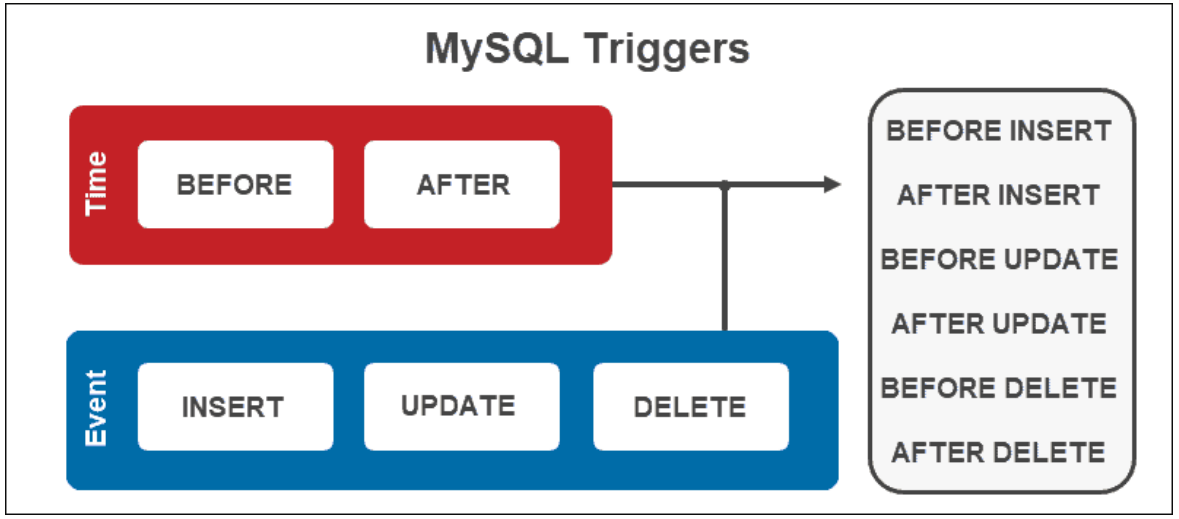
Query | test_trigger1

```
DELIMITER $$

CREATE
    /*[DEFINER = { user | CURRENT_USER }]*/
    TRIGGER `world`.`test_trigger1` BEFORE/AFTER INSERT/UPDATE/DELETE
    ON `world`.`<Table Name>`
    FOR EACH ROW BEGIN

    END$$

DELIMITER ;
```



Triggers (PBB_Colab DB): Trigger Log

```sql
-- 1. Create the database
CREATE DATABASE PBB_Colab;
USE PBB_Colab;
```

```sql
-- 2. Create the main table
CREATE TABLE housemates (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   age INT);
```

```sql
-- 3 Create the log table for the trigger
CREATE TABLE housemate_log (
   log_id INT AUTO_INCREMENT PRIMARY KEY,
   housemate_id INT,
   action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

```sql
-- 4 Create the trigger
DELIMITER $$
CREATE TRIGGER before_housemate_insert
BEFORE INSERT ON housemates
FOR EACH ROW
BEGIN
   INSERT INTO housemate_log (housemate_id)
   VALUES (NEW.id);
END
$$ DELIMITER ;
```

Triggers (PBB_Colab DB): Trigger Log

```
-- Remove safety updates when modifying tables
SET SQL_SAFE_UPDATES = 0;
```

```
-- 5 Use the trigger (insert a new housemate)
INSERT INTO housemates (name, age)
VALUES ('Brent Manalo', 25);

-- Check the table values
SELECT*from housemates;
SELECT*from housemate_log;
```

```
-- 6 Check existing triggers in this database
SHOW TRIGGERS FROM PBB_Colab;
```

id	name	age
1	Brent Manalo	25
NULL	NULL	NULL

log_id	housemate_id	action_time
1	0	2025-07-06 22:46:56
NULL	NULL	NULL

Trigger	Event	Table	Statement	Timing
before_housemate_insert	INSERT	housemates	BEGIN INSERT INTO housemate_log (housemate_id) VALUES (NEW.id); END	BEFORE

Triggers: Accessing OLD and NEW Values

- Triggers can use:
  - **NEW.column_name**: Value *after* UPDATE
  - **OLD.column_name**: Value *before* UPDATE
- Useful for:
  - Detecting specific changes
  - Auditing important status updates
- Example: In PBB_colab schema, make a “log” every time a housemate’s status changes from 'IN-HOUSE' → 'EVICTED

Triggers: Accessing OLD and NEW Values

- Which triggers can use NEW and OLD?

Trigger Event	NEW available?	OLD available?
INSERT	Yes ✔️	No 🚫
UPDATE	Yes ✔️	Yes ✔️
DELETE	No 🚫	Yes ✔️

Triggers: Accessing OLD and NEW Values

```
-- 1 Create the housemate table
CREATE TABLE housemate (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   status VARCHAR(20));
```

```
-- 2 Insert sample data
INSERT INTO housemate (name, age, status)
VALUES ('Charlie Fleming', 16, 'IN-HOUSE’);
SELECT * FROM housemate;
```

```
-- 3 Create the eviction history log table
CREATE TABLE housemate_eviction_history (
   history_id INT AUTO_INCREMENT PRIMARY KEY,
  housemate_id INT,
   old_status VARCHAR(20),
   new_status VARCHAR(20),
   action_time TIMESTAMP DEFAULT
CURRENT_TIMESTAMP);
```

```
-- 4 Create the trigger to log status changes
DELIMITER $$
CREATE TRIGGER log_eviction_change
AFTER UPDATE ON housemate
FOR EACH ROW
BEGIN
   IF OLD.status = 'IN-HOUSE' AND NEW.status = 'EVICTED' THEN
      INSERT INTO housemate_eviction_history (housemate_id,
                                              old_status, new_status)
      VALUES (OLD.id, OLD.status, NEW.status);
   END IF;
END
$$ DELIMITER ;
```

Triggers: Accessing OLD and NEW Values

```
-- 5 Simulate evicting Charlie
UPDATE housemate
SET status = 'EVICTED’
WHERE id = 1;

-- 6 View current housemates
SELECT * FROM housemate;

-- 7 View eviction history log
SELECT * FROM
housemate_eviction_history;
```

id	name	age	status
1	Charlie Fleming	16	EVICTED
NULL	NULL	NULL	NULL

history_id	housemate_id	old_status	new_status	action_time
1	1	IN-HOUSE	EVICTED	2025-07-06 23:54:39
NULL	NULL	NULL	NULL	NULL

Trigger	Event	Table	Statement	Timing	Created

log_eviction_change	UPDATE	housemate	BEGIN
			IF OLD.status = 'IN-HOUSE' AND NEW.status = 'EVICTED' THEN
			INSERT INTO housemate_eviction_history (housemate_id, old_status, new_status)
			VALUES (OLD.id, OLD.status, NEW.status);
			END IF;
			END
				AFTER	2025-07-06 23:54:36.01

Restricting Invalid Data with Triggers

- We're adding a **trigger to check** if the new status is valid.
- If not, **it stops** the insert with an error.

```
-- 1. Make sure the table is created
CREATE TABLE housemate (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   status VARCHAR(20)
);
```

```
-- 2. Create the Trigger
DELIMITER $$
CREATE TRIGGER check_status_before_insert
BEFORE INSERT ON housemate
FOR EACH ROW
BEGIN
   IF NEW.status NOT IN ('IN-HOUSE', 'EVICTED') THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid status. Must be In-House or Evicted';
   END IF;
END
$$ DELIMITER ;
```

```
-- 3. Test using an INCORRECT input value
INSERT INTO housemate (name, age, status)
VALUES ('Brent Manalo', 25, 'INSIDE');
```

```
69  00:31:42  INSERT INTO housemate (name, age, status) VALUES ('Brent Manalo', 25, 'INSIDE')
```

Error Code: 1644. Invalid status. Must be IN-HOUSE or EVICTED.

Checking existing triggers

1. To see all triggers in your current database

```
SHOW TRIGGERS;
```

2. To see triggers of a specific schema

```
SHOW TRIGGERS FROM PBB_Colab;
```

3. For detailed information on all triggers

```
SELECT * FROM INFORMATION_SCHEMA.TRIGGERS
WHERE TRIGGER_SCHEMA = 'PBB_Colab';
```

Demo: Deleting a row

- **Scenario**: When a housemate is evicted (deleted from the housemate table), you want to keep a record of them in an eviction_log table.

```
-- 1. Create housemate table
CREATE TABLE housemate (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   status VARCHAR(20)
);
```

```
-- 2. Create eviction_log table
CREATE TABLE eviction_log (
   eviction_id INT AUTO_INCREMENT PRIMARY KEY,
   housemate_id INT,
   name VARCHAR(50),
   age INT,
   status VARCHAR(20),
   eviction_date TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
);
```

```
-- 3. Create the trigger
DELIMITER $$
CREATE TRIGGER after_housemate_delete
AFTER DELETE ON housemate
FOR EACH ROW
BEGIN
   INSERT INTO eviction_log (housemate_id, name, age, status)
   VALUES (OLD.id, OLD.name, OLD.age, OLD.status);
END;
$$ DELIMITER ;
```

Demo: Deleting a row (Testing the trigger)

- **Scenario**: When a housemate is evicted (deleted from the housemate table), you want to keep a record of them in an eviction_log table.

```
-- 5. Add sample housemates
INSERT INTO housemate (name, age, status)
VALUES ('Charlie', 25, 'Active'),
        (‘Esnyr', 23, 'Active'),
        (‘River', 24, 'Active');
```

```
-- 6. Delete a housemate
SET SQL_SAFE_UPDATES = 0;
DELETE FROM housemate WHERE name = 'Charlie';
```

```
-- 7. Check the log table
SELECT * FROM eviction_log;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

eviction_id	housemate_id	name	age	status	eviction_date
1	1	Charlie	25	Active	2025-07-09 23:36:03
NULL	NULL	NULL	NULL	NULL	NULL